Common Pseudo-elements:

```
h1::before {
  content: '👉 ';
  color: orange;
}
h1::after {
  content: ' 🔥';
}
p::first-line{
  font-size: 2em;
  color: red;
}
::selection {
  background-color: yellow;
  color: black;
}
```

Common Pseudo-classes:

```
button:hover {
  background-color: lightblue;
}

input:focus {
  border-color: green;
}
li:nth-child(2) {
  color: red;
}
p:first-child {
  font-weight: bold;
}
a:active {
  color: red;
}
```

Pseudo-elements and Pseudo-classes Overview

In CSS, both **pseudo-elements** and **pseudo-classes** are used to style specific parts or states of elements, but they do this in different ways:

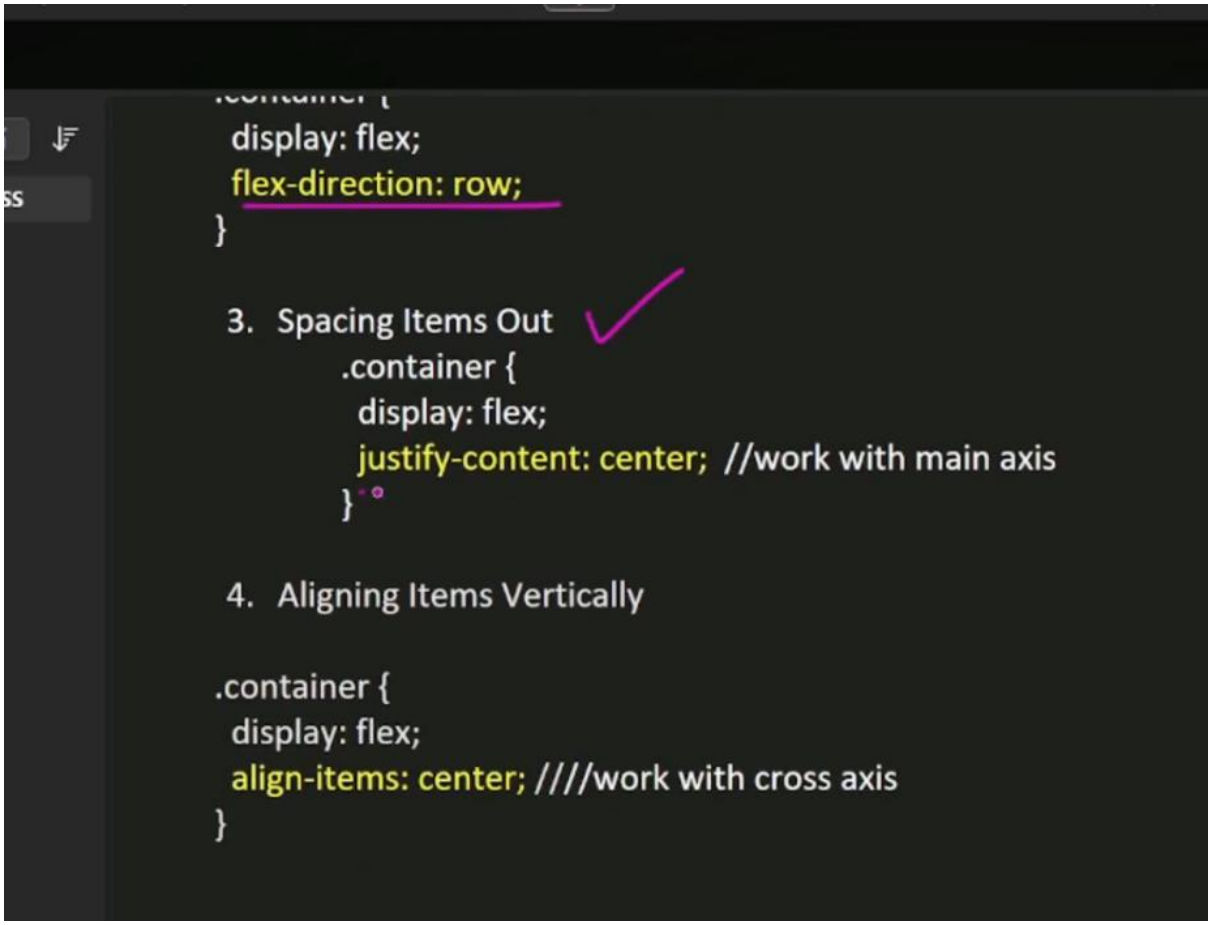
- **Pseudo-classes**: These apply styles to an element **based on its state or condition**, such as when an element is hovered over, focused, c even its position relative to siblings.
- **Pseudo-elements**: These allow you to **style specific parts** of an element, like the first letter, first line, or even inserting new content before or after the element's content.

Flex Item Properties

1. **order**: Controls the order of flex items. Items are displayed in ascending order of order values.
2. **flex-grow**: Defines how much space an item should take relative to others. A value of 1 makes an item grow to fill remaining spac
3. **flex-shrink**: Defines how much an item should shrink if there's not enough space. A value of 0 prevents shrinking.
4. **flex-basis**: Sets the default size of an item before the remaining space is distributed. Can be set in pixels, percentages, etc.
5. **align-self**: Allows individual items to override the align-items property and align themselves differently along the cross axis.

1. Handling Wrapping : If you have too many items to fit in a row, you might want them to wrap onto a new line rat
   than squish together.

```
.container {
 display: flex;
 flex-wrap: wrap; /* Now the items will wrap to a new line if needed */
}
```



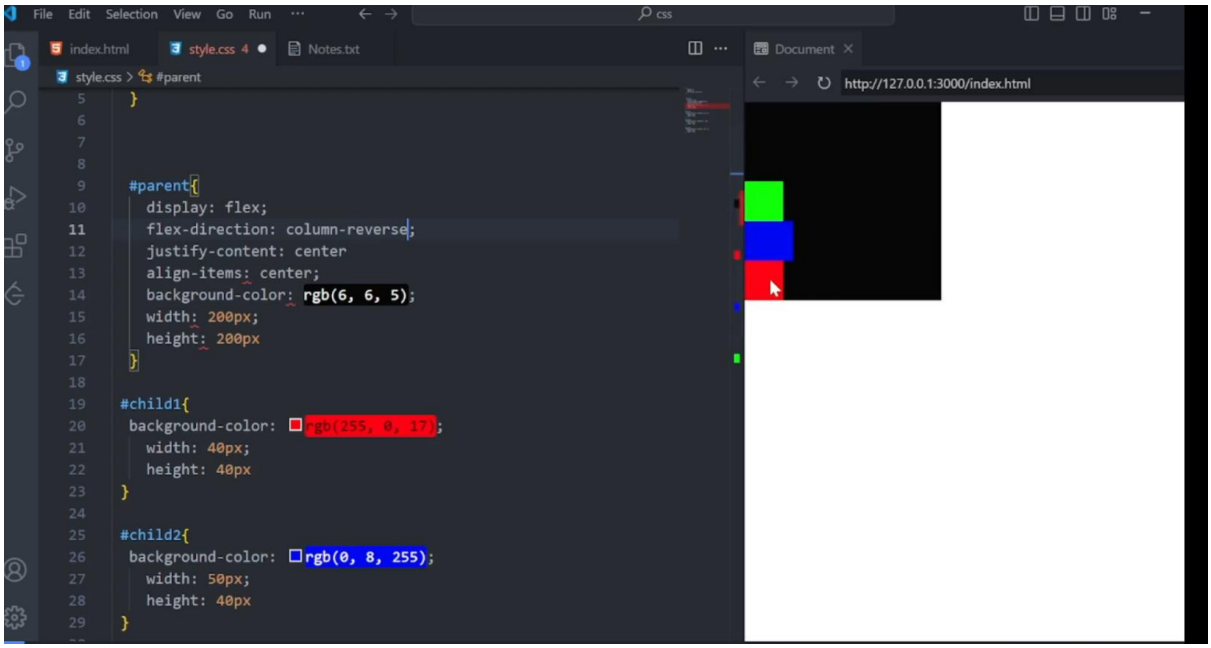
```
  display: flex;
  flex-direction: row;
}
```

3. Spacing Items Out

```
.container {
  display: flex;
  justify-content: center;  //work with main axis
}
```

4. Aligning Items Vertically

```
.container {
  display: flex;
  align-items: center; ////work with cross axis
}
```



Why Are Two Axes Needed?
These two axes are essential because Flexbox is designed to give you control over both the direction and alignment of items inside a container

2. Cross Axis
The cross axis is always perpendicular to the main axis. If the main axis is horizontal (in a row layout), the cross axis is vertical. If the main axis is vertical (in a column layout), the cross axis is horizontal.

In flex-direction: row, the cross axis runs vertically (top to bottom).
In flex-direction: column, the cross axis runs horizontally (left to right).

Main Axis: Controls the flow of items (horizontally or vertically, depending on flex-direction).
Cross Axis: Handles the alignment of items perpendicular to the main axis (vertically if in a row, horizontally if in a column).

Note : flex property will apply on parent not on child's

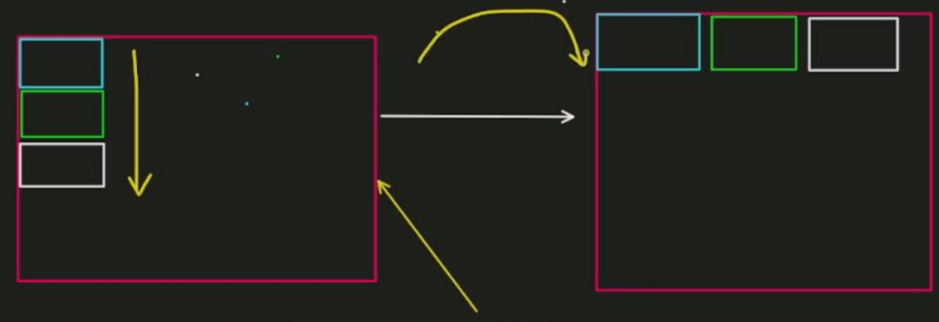
When you apply Flexbox, two axes get activated automatically: the main axis and the cross axis. These axes control ho the flex items are laid out inside the flex container

1. Main Axis
The main axis is the primary direction in which the flex items are laid out. By default, when you apply display: flex, the main axis runs horizontally from left to right (because flex-direction: row is the default).

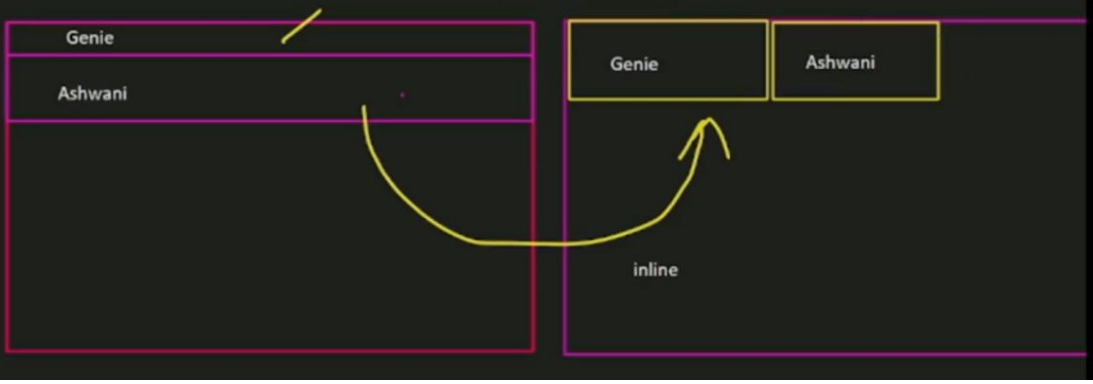
If you set flex-direction: row, the main axis is horizontal.
If you set flex-direction: column, the main axis becomes vertical.

2. Cross Axis
The cross axis is always perpendicular to the main axis. If the main axis is horizontal (in a row layout), the cross axis is vertical. If the main axis is vertical (in a column layout), the cross axis is horizontal.

Flex : when we want element side by side them flex is better choice as compare to inline, because we have more control over element



Note : flex property will apply on parent not on child's



Note: If an element is a block-level element, it takes up the entire width of the row, and no other elements can appear beside it—just like a lion in the jungle, ruling over its space alone (similar to how only one block-level element fits in a row).

but when we make block and inline-block the it will allow other elements can appear beside it

Inline elements other elements can appear beside it if these is space
on the other hand, are like monkeys—they're flexible and allow other elements to sit beside them in the same line, leaving space around them.

If you make any element inline then you cannot change it's width and height

Inline-block : this is will make block and inline-block

Display: it is very easy if you understand how it works

sets whether an element is treated as a block or inline box and the layout used for its children

What is block : in html each tag have their nature like inline or block just like div and span

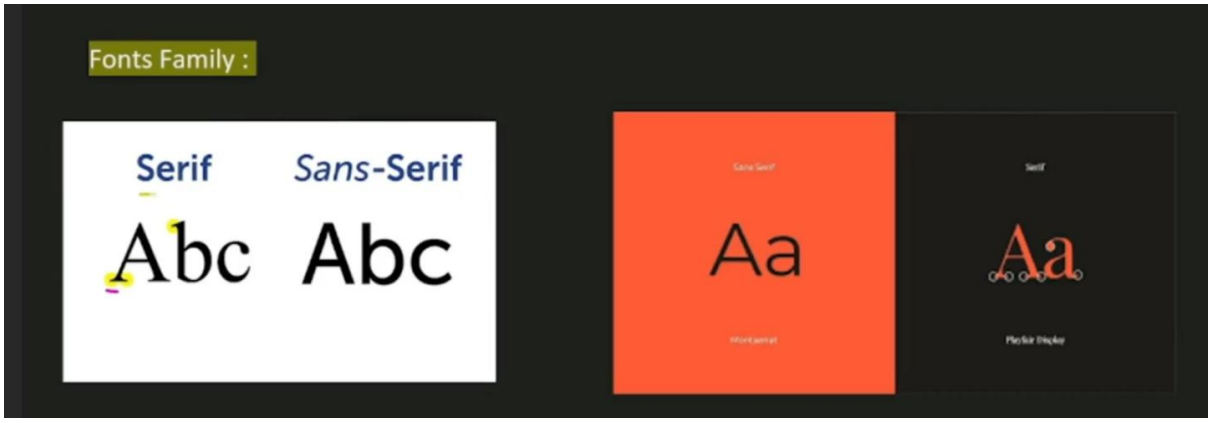
Eg: if you h1 and again h1 both will come one by one it's called block nature

Properties with fonts:

```
font-family: Gowun Batang;
font-weight: 600;
text-transform: capitalize/uppercase;
text-decoration: line-through;
font-size: 20px/1vw;
```

```
Line-height
-----------
Using this tag each line will follow this unit
```

```
Text-align
----------
Used to align text on screen
Note : text-align always start from left side by default
```



width & height - defines width and height of element

color – defines the text color of an element.

bgcolor - defines background color

units - px % vh vw em rem define lengths, sizes, and measurements:

font-family – specifies the typeface or font to be used in an element.

font-size – controls the size of the text in an element.

line-height – sets the space between lines of text (leading).

text-align – aligns the text horizontally (left, right, center, justify).

padding – defines the space between the content and the element's border (inside spacing).

margin – sets the space outside the element's border (outside spacing).

border – defines the style, width, and color of an element's border.

index.html > html > head > style > p

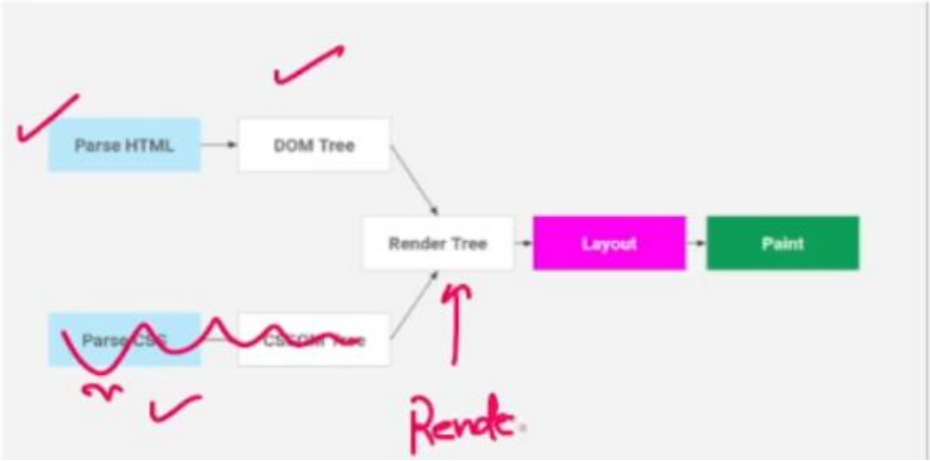
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        p {
            color: aqua;
        }
    </style>
</head>
<body>
    <p style="color: rebeccapurple;">Lorem, ipsum dolor sit amet consectetur adipisicing elit. S

</body>
</html>
```

```
Non-semantic elements : div span
semantic elements: <header>,<nav>,<section>,<article>,<footer> etc
```

💡 Skills to Add Before Placements

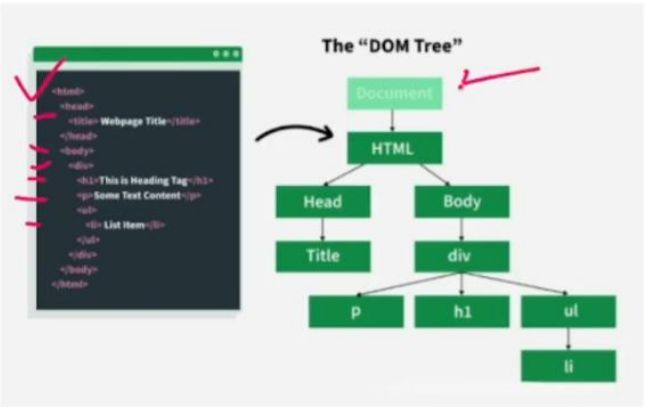
To stand out, consider adding:

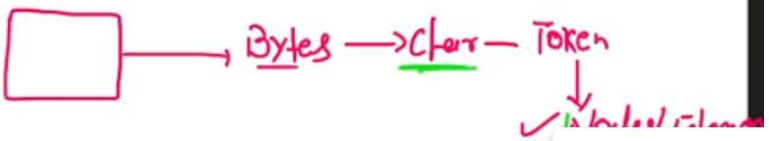
- **Kafka** or **Redis** (seen in almost all Spring Boot JDs)
- **JUnit + Mockito** (testing — frequently asked)
- **AWS basics** (EC2, S3) — very common in JDs
- **Kubernetes** basics (bonus)



Bytes => Characters => Tokens => Node => DOM

ELEMENTS

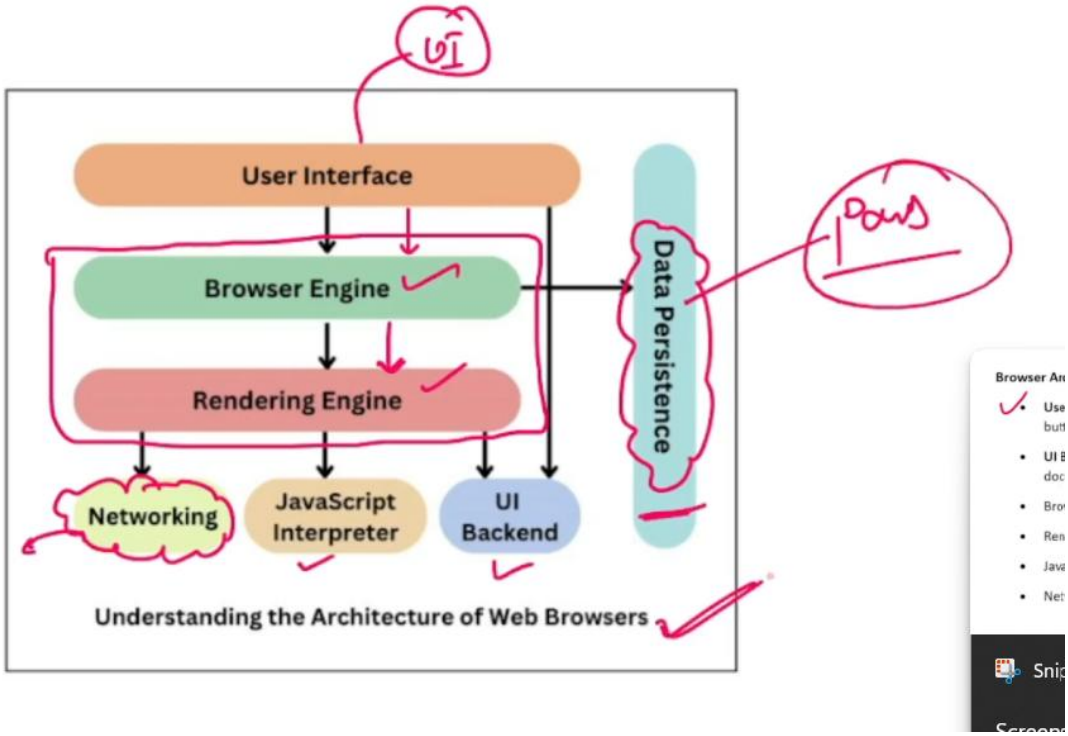




How HTML works in browser?

Bytes => Characters => Tokens => Node => DOM

ELEMENTS

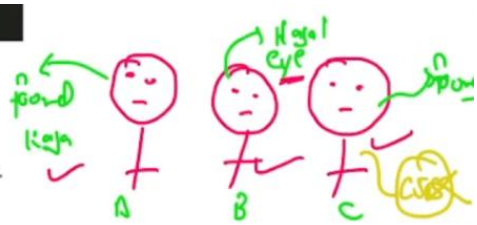


Browser Architecture

- **User Interface**: Provides the UI for the browser, including the title bar, menu bar, shortcut buttons, address bar, extensions, favorites, etc.
- **UI Backend**: Stores the data required for the browser, including user credentials, recently viewed documents, etc.
- Browser Engine: Translates HTML and CSS.
- Rendering Engine: Responsible for generating output.
- JavaScript Interpreter: Translates JavaScript line by line.
- Network: Tracks the performance of a page.

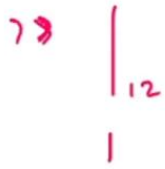
Evolution of HTML

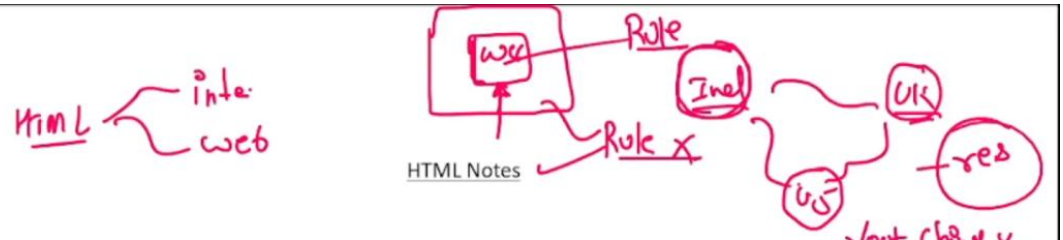
- **CERN Labs** developed a language called **GML (Generic Markup Language)** for the Internet.
- **CERN** improved GML with more features and introduced it as **SGML (Standard Generic Markup Language)**.
- In the early **1990s**, **Tim Berners-Lee** introduced **HTML**.
- **IETF (Internet Engineering Task Force)** developed HTML up to **version 3.1**.
- In early **2004**, **WHATWG (Web HyperText Application Technology Work Group)** took responsibility for HTML and started evolving it.
- **WHATWG** started with **HTML 4**, and the latest version is **HTML 5 (2014)**.



What is HTML?

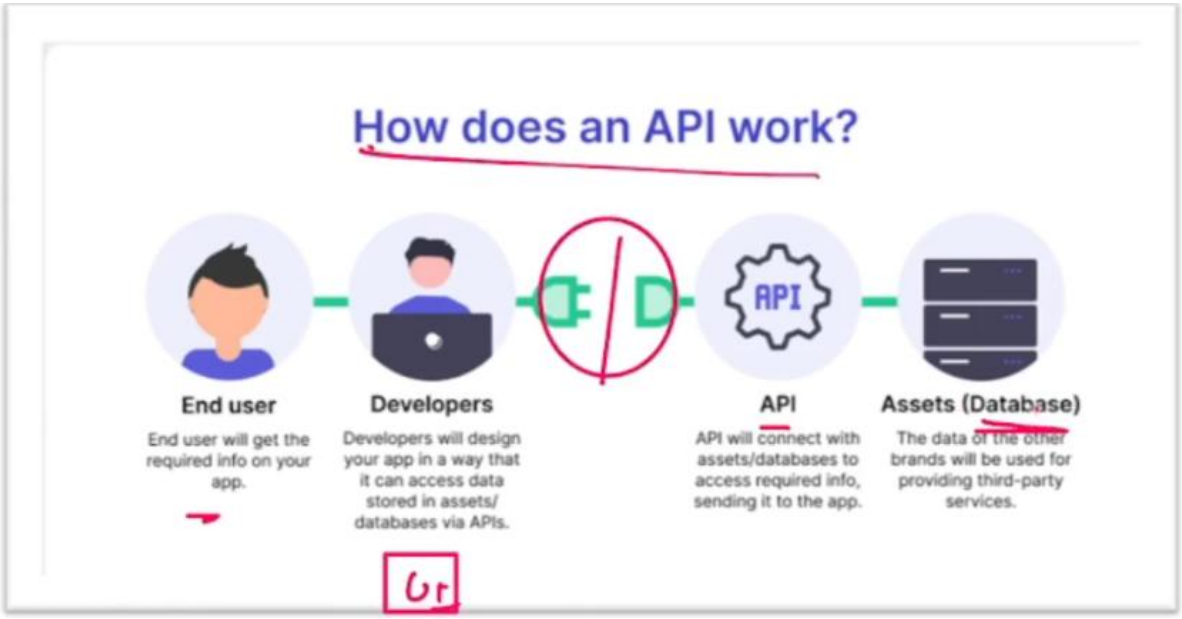
- HTML is not programming language it is markup language.
- HTML stands for HyperText Markup Language.
- HyperText refers to text that takes the user beyond the content they view on the screen.
- Markup is a term derived from "Marking up," which refers to presentation.
- Markup language is a language used for presentation.
- HTML is a presentation language.





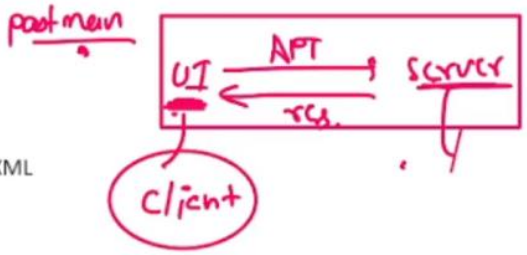
What is Web?

- Internet is a wide area network that connects computers all over the world.
- In the early 1990s, **Tim Berners-Lee** introduced the concept of the "Web".
- Web is a portion of the internet with restricted access.
- W3C [World Wide Web Consortium] maintains the standards of web.
- The latest version of web is web-3

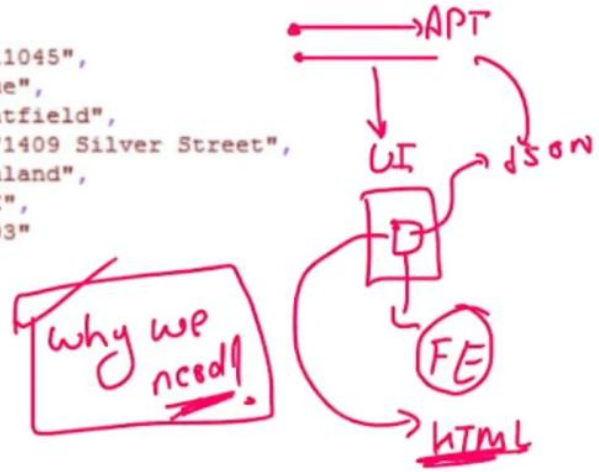


What is API?

1. API: Application programming interface
2. It provides data to client into format of JSON & XML

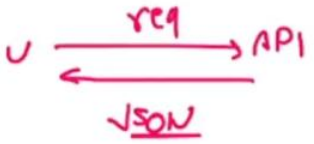



```
            "trackingno": "TN0039291",
            "custid": "11045",
            "customer": [
                {
                    "custid": "11045",
                    "fname": "Sue",
                    "lname": "Hatfield",
                    "address": "1409 Silver Street",
                    "city": "Ashland",
                    "state": "NE",
                    "zip": "68003"
                }
            ]
        }
    ]
}
```

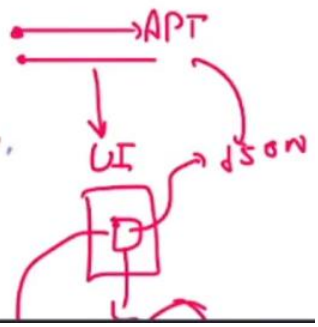



Role of frontend developer

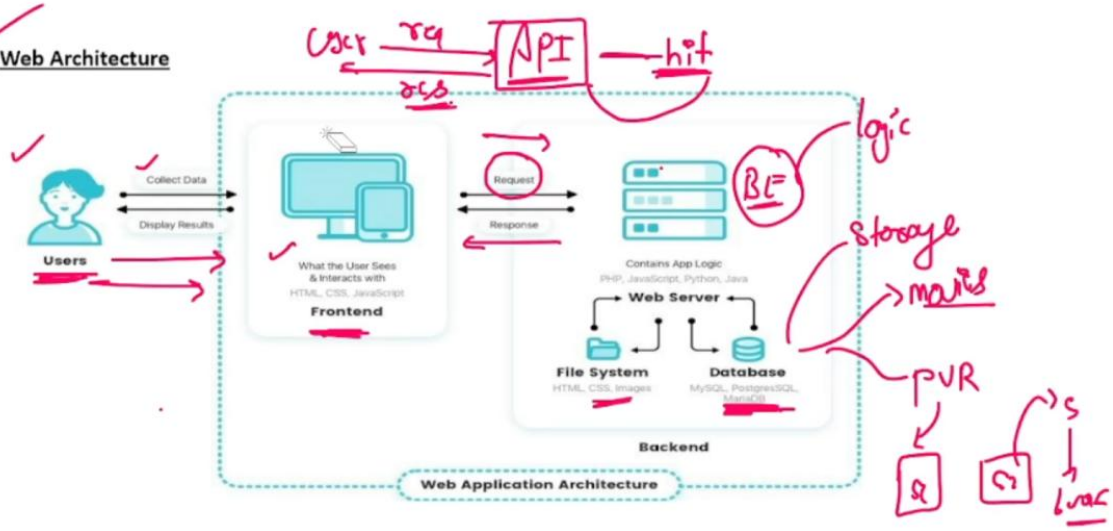
1. Making web application
2. Transforming JSON formatted data to user understanding data



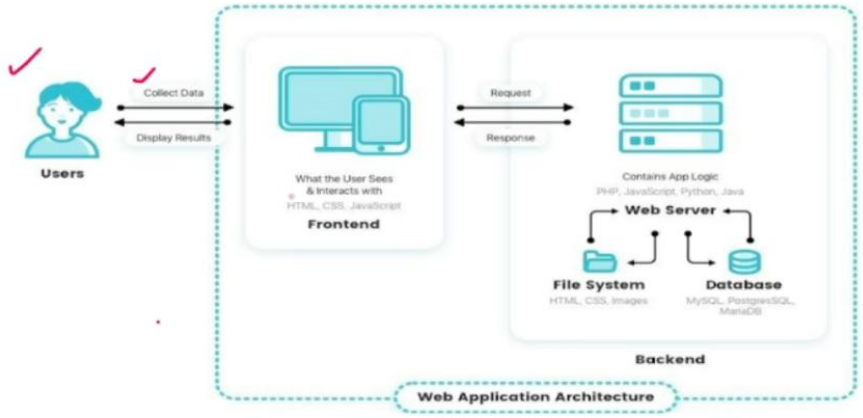

```
{
    "orders": [
        {
            "orderno": "748745375",
            "date": "June 30, 2088 1:54:23 AM",
            "trackingno": "TN0039291",
            "custid": "11045",
            "customer": [
                {
                    "custid": "11045",
                    "fname": "Sue",
                    "lname": "Hatfield",
                    "address": "1409 Silver Street",
                    "city": "Ashland",
                    "state": "NE",
                    "zip": "68003"
                }
            ]
        }
```

Web Architecture



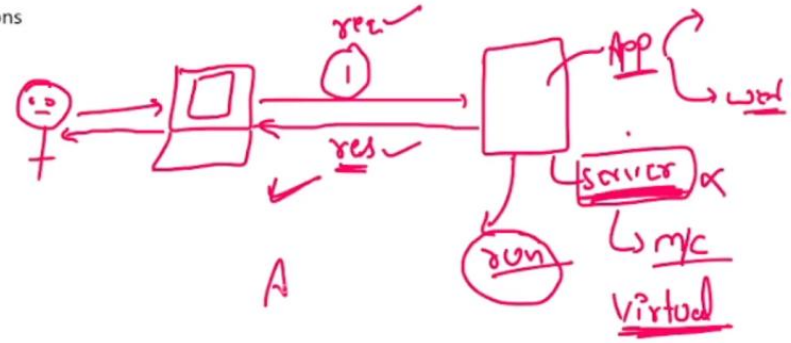
Web Architecture



- Building Web Applications

Types of Applications:

1. Desktop
2. Web
3. Distributed
5. Mobile



Full Stack Developer

Developer who knows ui(HTML + CSS + JS + React) + BE(Java + Spring Boot)

Web Architecture